

Guía del Frontend — Krypton E-commerce

Guía didáctica de **cómo funciona el frontend** (la tienda que ve el usuario) y **cómo le pide datos al backend**. Pensada para que cualquiera del equipo la entienda.

1. Qué es y con qué está hecho

El frontend es una **SPA** (Single Page Application): una sola página web que cambia de vista sin recargar, hablándole al backend por HTTP. Es el lado visible — catálogo, carrito, checkout, panel de administración.

Pieza	Tecnología
Librería UI	React 19
Bundler / dev server	Vite
Lenguaje	TypeScript
Ruteo	React Router 7
HTTP	Axios
Gráficos (reportes)	Recharts

El backend y el frontend son **proyectos separados**: el front no sabe nada de Java, solo conoce las **URLs** y la **forma del JSON** que el backend expone (ver guía del backend, sección 7).

2. Estructura del proyecto

```
frontend/src/
├─ main.tsx      Punto de entrada: monta React + los Providers (Auth, Cart, Router)
├─ App.tsx       El mapa de rutas (qué URL muestra qué página)
├─ config.ts     La URL del backend (VITE_API_BASE_URL)
├─
├─ lib/          Utilidades de infraestructura
│  └─ api.ts      Cliente Axios + interceptor que adjunta el token JWT
│  └─ apiError.ts Extrae el mensaje de error que manda el backend
│  └─ download.ts Descarga de archivos (PDF/Excel)
├─
├─ auth/         Autenticación
│  └─ AuthContext.tsx Estado global de sesión (token, usuario, login/logout)
│  └─ RequireAdmin.tsx Guard: protege las rutas /admin
├─
├─ cart/
│  └─ CartContext.tsx Estado global del carrito
├─
├─ models/       Tipos TypeScript que ESPEJAN los DTOs del backend
│  └─ auth.ts product.ts order.ts cart.ts report.ts
├─
├─ features/     Una carpeta por dominio (cada una = páginas + su capa API)
│  └─ home catalog cart checkout orders auth legal admin
├─
├─ components/   Componentes reutilizables (ProductCard, Navbar, Footer...)
├─ app/          MainLayout.tsx (el "marco" con navbar + footer)
└─ styles/       Design system: tokens de color, tipografía, spacing
```

La organización es **por feature**: cada dominio (catálogo, carrito, admin...) tiene su carpeta con sus páginas y su archivo `*.api.ts` que habla con el backend. Todo lo de un tema vive junto.

3. Cómo le habla al backend

Este es el puente. Todo pasa por **un único cliente Axios** y un patrón repetido.

a) La URL base — `config.ts`

```
export const API_BASE_URL = import.meta.env.VITE_API_BASE_URL ?? 'http://localhost:8080';
```

En desarrollo apunta a `localhost:8080`. En producción se setea `VITE_API_BASE_URL` al deploy.

b) El cliente Axios + el token — `lib/api.ts`

```
export const api = axios.create({ baseURL: API_BASE_URL });

// Antes de CADA request, si hay token guardado, lo adjunta:
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

Gracias a este **interceptor**, ninguna feature tiene que preocuparse por el token: se agrega automáticamente a toda llamada.

c) El patrón `*.api.ts` por feature

Cada feature expone funciones que envuelven los endpoints del backend. Ejemplos reales:

```
// catalog/products.api.ts
search(filter, page, size)      → GET  /api/products      (paginado + filtros)
getById(id)                    → GET  /api/products/{id}

// cart/cart.api.ts
getCart()                      → GET  /api/cart
addItem({productId, quantity}) → POST /api/cart/items
updateItem(itemId, quantity)   → PUT  /api/cart/items/{itemId}
removeItem(itemId)             → DELETE /api/cart/items/{itemId}

// orders/orders.api.ts
checkout(body)                  → POST /api/orders/checkout
getMyOrders()                   → GET  /api/orders
payOrder(id, body)              → POST /api/orders/{id}/pay
downloadMyComprobante(id)       → GET  /api/orders/{id}/comprobante (PDF)

// admin/admin-products.api.ts
createProduct(body)             → POST  /api/admin/products
updateProduct(id, body)         → PUT   /api/admin/products/{id}
deleteProduct(id)               → DELETE /api/admin/products/{id}
uploadImage(productId, file)    → POST  /api/admin/products/{id}/images (multipart)
```

Las páginas **nunca llaman a Axios directo**: usan estas funciones. Así, si cambia un endpoint, se toca un solo archivo.

d) Errores y descargas

- `lib/apiError.ts` → `apiErrorMessage(err, fallback)` extrae el campo `error` que manda el backend (ej. *"La categoría tiene productos asociados"*) para mostrarlo al usuario.
- `lib/download.ts` → `saveBlob(blob, filename)` crea un `<a>` temporal para bajar PDFs/Excel. Las descargas usan `responseType: 'blob'` y el token va automático (interceptor).

e) Los `models/` espejan el JSON

Los tipos en `src/models/` describen exactamente la forma del JSON que devuelve el backend (campos en inglés, camelCase — el "contrato"). Ej: `ProductResponse`, `OrderResponse`, `CartResponse`, `PageResponse<T>` (envoltorio de paginación), y los enums `OrderStatus`, `PaymentMethod`, `DocumentType`.

Estos nombres TypeScript son del front; coinciden con el backend en los **campos**, no en el nombre de clase (el backend los llama `ProductoResponse` , etc., pero el JSON es el mismo).

4. Autenticación en el front — `auth/AuthContext.tsx`

El estado de sesión es **global** (un Context de React), accesible con el hook `useAuth()` :

```
const { user, isAuthenticated, isAdmin, login, register, logout } = useAuth();
```

Cómo funciona: - **Login:** `POST /api/auth/login` → recibe `{ token, ... }` → guarda el token en `localStorage` → **decodifica** el JWT (sin validar la firma; eso es del backend) para sacar `email` , `role` y `exp` → setea el `user` . - **Registro:** `POST /api/auth/register` y luego auto-login con las mismas credenciales. - **Logout:** borra el token de `localStorage` y limpia el `user` . - **Al cargar la app:** si hay un token guardado pero ya **expiró**, lo descarta. - `isAdmin` = el `role` del token es `ADMIN` .

El guard de admin — `auth/RequireAdmin.tsx`

Envuelve las rutas `/admin` . Si no hay sesión → redirige a `/cuenta/ingresar` ; si hay sesión pero no es admin → redirige a `/` . **Es solo para la experiencia de usuario:** la seguridad real la impone el backend, que valida `ROLE_ADMIN` en cada endpoint `/api/admin/**` (el front no puede "colarse" aunque trucara esto).

5. Estado global: el carrito — `cart/CartContext.tsx`

El carrito **vive en el backend** (persistido por usuario); el front lo refleja. Con `useCart()` :

```
const { cart, itemCount, loading, addItem, updateItem, removeItem, clear, refresh } = useCart();
```

- Al **iniciar sesión** → `refresh()` trae el carrito del backend (`GET /api/cart`).
- Al **cerrar sesión** → el carrito se limpia.
- `addItem` / `updateItem` reciben el carrito completo de vuelta y lo setean; `removeItem` / `clear` (que responden 204 sin cuerpo) disparan un `refresh()` .

Así el ícono del carrito en la navbar (`itemCount`) siempre está sincronizado.

6. Ruteo y "shells" — `App.tsx`

Las URLs se mapean a páginas. Hay dos "marcos" (shells) visuales:

```
Shell "auth" (sin navbar/footer, pantalla inmersiva)
/cuenta/ingresar → LoginPage
/cuenta/registro → RegisterPage

Shell "tienda" (MainLayout: navbar + footer)
/ → HomePage
/catalogo → CatalogPage /catalogo/:id → ProductDetailPage
/carrito → CartPage
/checkout → CheckoutPage
/pedidos → OrdersPage /pedidos/:id → OrderDetailPage
/terminos /privacidad → páginas legales

/admin (protegido por <RequireAdmin>) → AdminLayout (sidebar)
/admin/productos · /admin/categorias · /admin/usuarios · /admin/pedidos · /admin/reportes

* (cualquier otra) → redirige a /
```

7. Las features, una por una

Tienda (cliente)

- **home** — `HomePage` : portada con hero, categorías, productos destacados. Se puede agregar al carrito desde las cards (pide sesión).
- **catalog** — `CatalogPage` : grilla paginada (12/pág) con filtros en una barra lateral (búsqueda, categoría, rango de precio). Los filtros van en la URL (`?q=...&categoryId=...`), así el link es compartible. `ProductDetailPage` : detalle con galería de imágenes.
- **cart** — `CartPage` : revisar/editar el carrito (cambiar cantidades, quitar líneas) con el resumen (subtotal, envío, IGV, total). Si no hay sesión, muestra un aviso para entrar.
- **checkout** — `CheckoutPage` : datos del comprobante (BOLETA/FACTURA, nombre, documento) y "Confirmar pedido" → crea la orden → lleva al detalle del pedido.
- **orders** — `OrdersPage` : historial de pedidos. `OrderDetailPage` : detalle + línea de tiempo del estado; si está PENDIENTE, botón "Pagar"; si está pagado, "Descargar comprobante" (PDF).
- **auth** — `LoginPage` / `RegisterPage` .
- **legal** — Términos y Privacidad (páginas estáticas).

Administración (`features/admin/` , solo ADMIN)

`AdminLayout` muestra un sidebar con 5 secciones. Cada página usa la API admin:

- **AdminProductsPage** — tabla de productos (buscar + filtrar por categoría), crear/editar (modal), y gestión de **imágenes** (subir, reordenar, marcar portada, borrar).
- **AdminCategoriesPage** — CRUD de categorías. No deja borrar una categoría con productos (409).
- **AdminOrdersPage** — pedidos paginados, filtro por estado y fechas, ver detalle (modal), **cambiar estado** (respetando las transiciones válidas) y descargar comprobante.
- **AdminUsersPage** — usuarios: crear, cambiar rol (CLIENTE/ADMIN) y activar/desactivar.
- **AdminReportsPage** — filtros por fecha y granularidad (día/mes); KPIs (total facturado, nº de órdenes, ticket promedio); gráficos con **Recharts** (ventas, top productos, kardex) y botones para exportar a **Excel/PDF**.

8. Estilos — `styles/design-system/`

Los estilos se basan en **tokens** (variables CSS) para mantener coherencia de marca:

- **colors.css** — paleta Krypton (navy, blue, yellow, orange) + colores semánticos (`--color-brand` , `--color-accent` , `--surface-*` , `--text-*` , `--action-*`).
- **typography.css** — fuente **Kanit**, escala de tamaños (display → xs), pesos y line-heights.
- **spacing.css** — escala de espaciado (base 4px), radios, sombras y duraciones de animación.

Cada feature tiene su `.css` que usa esas variables (`var(--surface-card)` , `var(--action-cta)` ...), así que un cambio de marca se hace en un solo lugar.

9. Convenciones que se repiten

- **Paginación**: el backend devuelve `PageResponse<T>` (`content` , `page` , `size` , `totalElements` , `totalPages`); la página base es 0.
- **Loading / error / empty**: las páginas manejan spinners de carga, mensajes de error (vía `apiErrorMessage`) y estados vacíos con CTA.
- **Debounce**: las búsquedas (catálogo, tablas admin) esperan ~350ms tras dejar de tipear para no disparar un fetch por tecla.
- **Confirmaciones**: las eliminaciones críticas piden confirmación.

- **Formato:** moneda en `PEN` (`Intl.NumberFormat('es-PE')`), fechas ISO del backend.
- **Imágenes:** si un producto no tiene imagen, se usa un placeholder de marca.
- **Idioma:** toda la UI está en **español**.

10. Cómo levantarlo

```
cd frontend
npm install      # solo la primera vez (instala React, Vite, etc.)
npm run dev      # arranca en http://localhost:5173
```

Necesita el **backend corriendo** en `http://localhost:8080` (ver guía del backend). Si el backend no está, las llamadas fallan y las páginas muestran error.

Build de producción: `npm run build` (genera estáticos en `frontend/dist/`).

Diseño y marca: ver el design system en `src/styles/design-system/`. Contrato con el backend: ver `guia-backend.md`, sección 7.